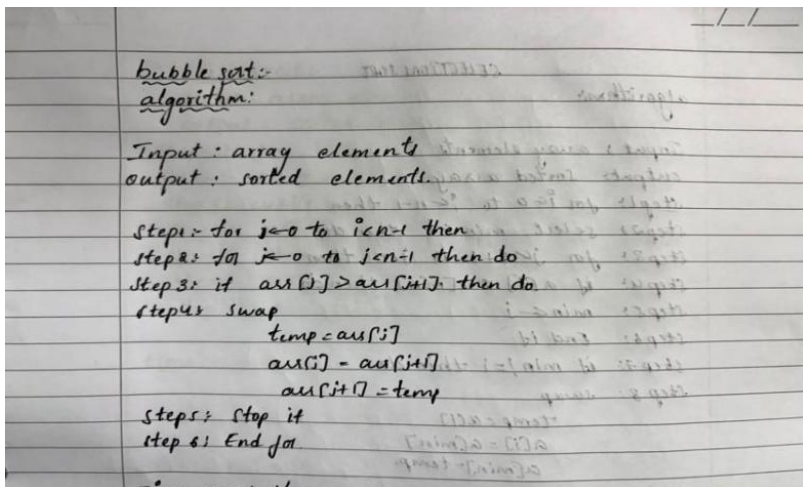


PRACTICAL-1

AIM: Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort

BUBBLE SORT

ALGORITHM:



CODE:

```

#include <iostream>
using namespace std;

class bubblesort {
    int arr[100], size, i, j, temp;

public:
    void getinput() {
        cout << "Enter the size of the array: ";
        cin >> size;
        cout << "Enter the elements: ";
        for (i = 0; i < size; i++) {
            cin >> arr[i];
        }
    }

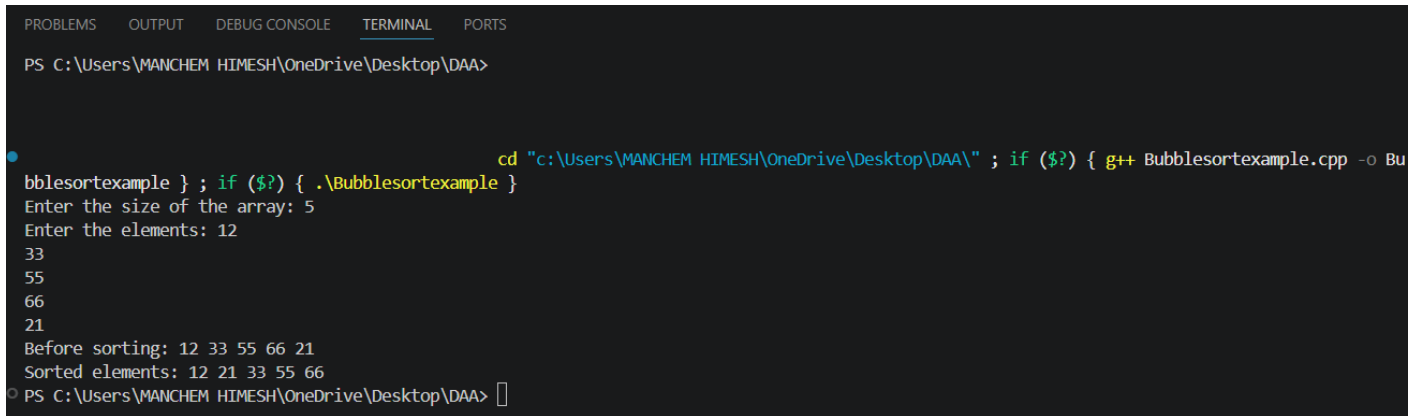
    void printelemets() {
        for (i = 0; i < size; i++) {
            cout << arr[i] << " ";
        }
        cout << endl;
    }

    void input() {
        for (i = 0; i < size - 1; i++) {
            for (j = 0; j < size - i - 1; j++) {
  
```

```
if (arr[j] > arr[j + 1]) {  
    temp = arr[j];  
    arr[j] = arr[j + 1];  
    arr[j + 1] = temp;  
}  
}  
}  
}  
};
```

```
int main() {  
    bubblesort bubble;  
    bubble.getinput();  
  
    cout << "Before sorting: ";  
    bubble.printelemets();  
  
    bubble.input();  
  
    cout << "Sorted elements: ";  
    bubble.printelemets();  
  
    return 0;  
}
```

OUTPUT:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS  
PS C:\Users\MANCHEM HIMESH\OneDrive\Desktop\DAA>  
  
cd "c:\Users\MANCHEM HIMESH\OneDrive\Desktop\DAA\" ; if ($?) { g++ Bubblesortexample.cpp -o Bu  
bblesortexample } ; if ($?) { .\Bubblesortexample }  
Enter the size of the array: 5  
Enter the elements: 12  
33  
55  
66  
21  
Before sorting: 12 33 55 66 21  
Sorted elements: 12 21 33 55 66  
PS C:\Users\MANCHEM HIMESH\OneDrive\Desktop\DAA>
```

TIME COMPLEXITY:

BUBBLE SORT

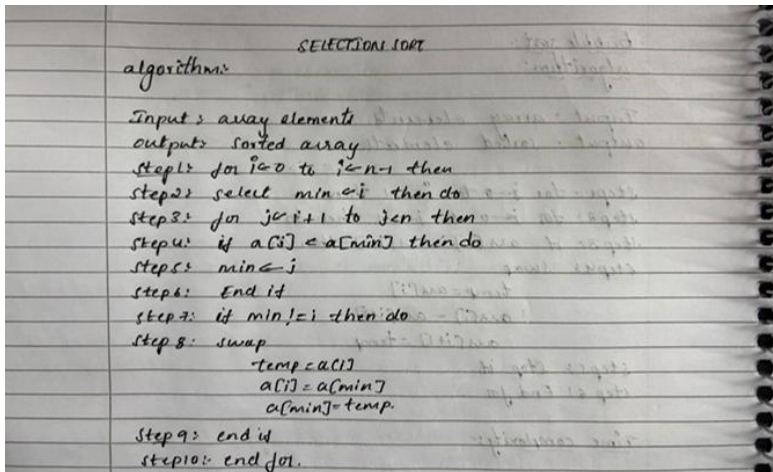
BEST CASE:
 first pass $\rightarrow n-1$
 now swapping
 loops stops
 $T(n) = n-1$
 Best case $= O(n)$

Average:
 for $(i=0; i < n-1; i++)$ $\rightarrow n-1$
 for $(j=0; j < n-i-1; j++)$ $\rightarrow n-1$
 if $(arr[i] > arr[j+1])$ $\rightarrow 1$
 temp $= arr[i]$
 $arr[i] = arr[j+1]$
 $arr[j+1] = temp$
 $T(n) = (n-1)(n-1) + 1 + 1 + 1 + \dots + 1$
 $T(n) = (n-1)(n-1) + (n-1)$
 $T(n) = n^2 - 2n + 1 + n - 1$
 $T(n) = n^2 - n$
 $\therefore O(n^2)$

worst case:
 for $(i=0; i < n-1; i++)$
 for $(j=0; j < n-i-1; j++)$
 if $(arr[i] > arr[j+1])$ $\rightarrow 1$
 temp $= arr[i]$
 $arr[i] = arr[j+1]$
 $arr[j+1] = temp$
 $T(n) = (n-1)(n-1) + 1 + 1 + 1 + \dots + 1$
 $T(n) = (n-1)(n-1) + (n-1)$
 $T(n) = n^2 - 2n + 1 + n - 1$
 $T(n) = n^2 - n$
 $\therefore O(n^2)$

SELECTION SORT

ALGORITHM:



CODE:

```

#include <iostream>
using namespace std;
class bubblesort
{
    int arr[100], n, i, j, temp, min;
public:
    void getinput()
    {
        cout << "Enter the size of the array: ";
        cin >> n;

        cout << "Enter the elements: ";
        for (i = 0; i < n; i++)
        {
            cin >> arr[i];
        }
    }
    void printelemets()
    {
        for (i = 0; i < n; i++)
        {
            cout << arr[i] << " ";
        }
        cout << endl;
    }
    void input()
    {
        for (i = 0; i < n - 1; i++)
        {
            min = i;
        }
    }
}
    
```

```
for (j = i+1; j < n; j++)
{
    if (arr[j] < arr[min])
    {
        min = j;
    }
}
if (min != i)
{
    temp = arr[i];
    arr[i] = arr[min];
    arr[min] = temp;
}
}
};
int main()
{
    bubblesort bubble;

    bubble.getinput();

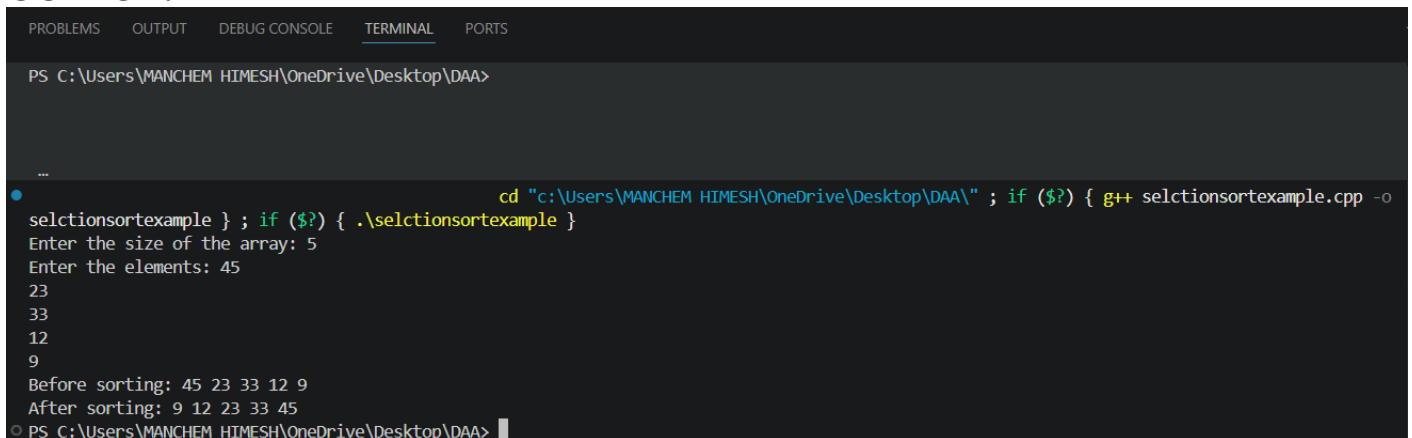
    cout << "Before sorting: ";
    bubble.printelemets();

    bubble.input();

    cout << "After sorting: ";
    bubble.printelemets();

    return 0;
}
```

OUTPUT:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\MANCHEM HIMESH\OneDrive\Desktop\DAA>

...

cd "c:\Users\MANCHEM HIMESH\OneDrive\Desktop\DAA\" ; if ($?) { g++ selctionsortexample.cpp -o
selctionsortexample } ; if ($?) { .\selctionsortexample }
Enter the size of the array: 5
Enter the elements: 45
23
33
12
9
Before sorting: 45 23 33 12 9
After sorting: 9 12 23 33 45
PS C:\Users\MANCHEM HIMESH\OneDrive\Desktop\DAA>
```

TIME COMPLEXITY:

SELECTION SORT

best case:

```

for (i=0; i<n-1; i++) → n-1
{
    min=i;
    for (j=i+1; j<n; j++) → n
    {
        if (arr[j] < arr[min]) → 1
        {
            min=j;
        }
    }
    temp = arr[i]; → 1
    arr[i] = arr[min]; → 1
    arr[min] = temp; → 1
}
            
```

$T(n) = n(n-1)$
 $= n^2 - n$
 $= O(n^2)$

average case:

```

for (i=0; i<n-1; i++) → n-1
{
    min=i;
    for (j=i+1; j<n; j++) → n
    {
        if (arr[j] < arr[min]) → 1
        {
            min=j;
        }
    }
    temp = arr[i]; → 1
    arr[i] = arr[min]; → 1
    arr[min] = temp; → 1
}
            
```

$T(n) = n(n-1)$
 $= n^2 - n$
 $= O(n^2)$

SELECTION SORT

worst case:

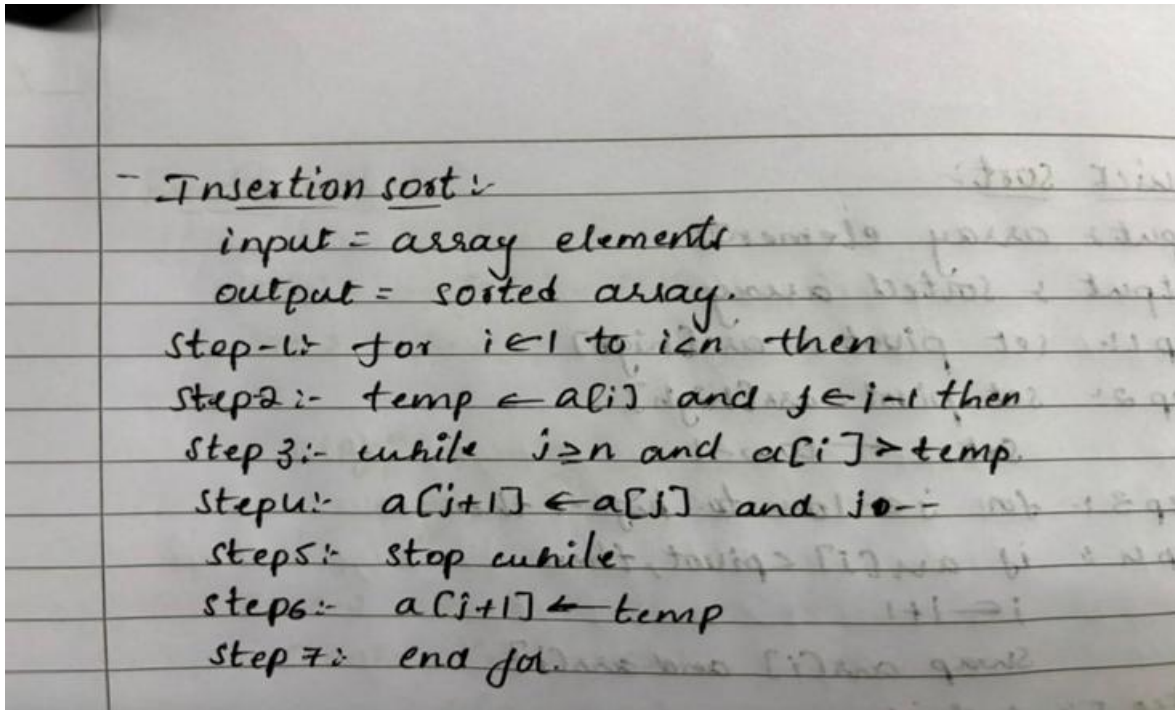
```

for (i=0; i<n-1; i++) → n-1
{
    min=i;
    for (j=i+1; j<n; j++) → n
    {
        if (arr[j] < arr[min]) → 1
        {
            min=j;
        }
    }
    temp = arr[i]; → 1
    arr[i] = arr[min]; → 1
    arr[min] = temp; → 1
}
            
```

$T(n) = n(n-1)$
 $= n^2 - n$
 $= O(n^2)$

INSERTION SORT

ALGORITHM:



- Insertion sort:-
input = array elements
output = sorted array.
step-1:- for $i \leftarrow 1$ to $i \leftarrow n$ then
step2:- $temp \leftarrow a[i]$ and $j \leftarrow i-1$ then
step3:- while $j \geq 1$ and $a[j] > temp$
step4:- $a[j+1] \leftarrow a[j]$ and $j \leftarrow j-1$
step5:- stop while
step6:- $a[j+1] \leftarrow temp$
step7:- end for.

CODE:

```
#include <iostream>
using namespace std;

class bubblesort
{
    int arr[100], n, i, j, temp, min;

public:
    void getinput()
    {
        cout << "Enter the size of the array: ";
        cin >> n;

        cout << "Enter the elements: ";
        for (i = 0; i < n; i++)
        {
            cin >> arr[i];
        }
    }
    void printelemets()
    {
        for (i = 0; i < n; i++)
        {
            cout << arr[i] << " ";
        }
    }
}
```

```
cout << endl;
}
void input()
{
    for (i = 1; i < n; i++)
    {
        temp=arr[i];
        j=i-1;
        while (j>=0 && arr[j]>temp){
            arr[j+1]=arr[j];
            j--;
        }
        arr[j+1]=temp;
    }
}
};
int main()
{
    bubblesort bubble;

    bubble.getinput();

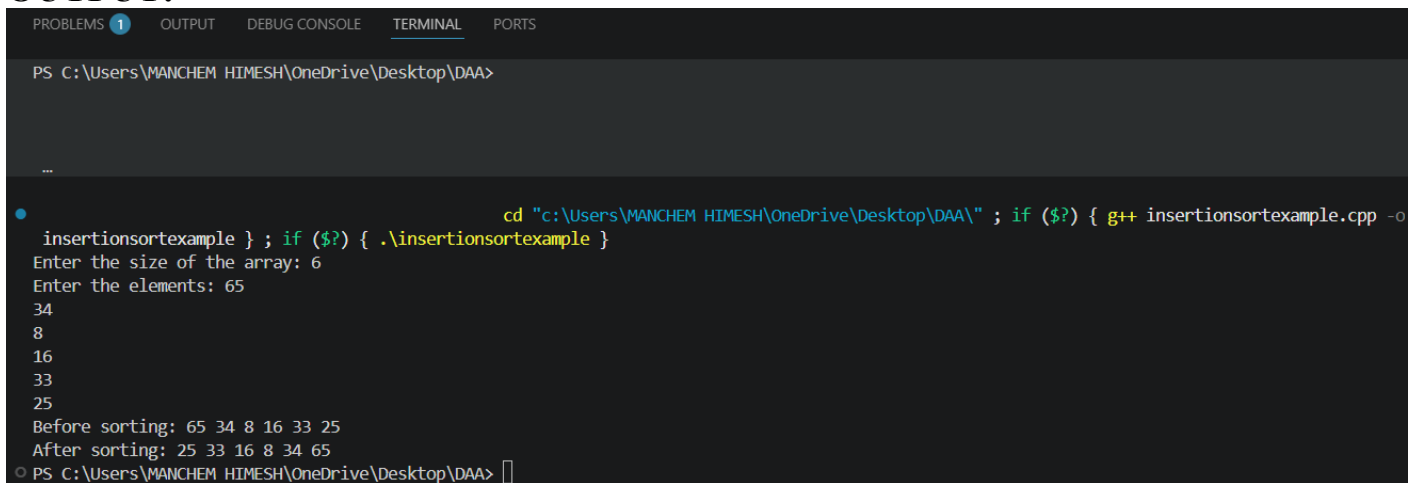
    cout << "Before sorting: ";
    bubble.printelemets();

    bubble.input();

    cout << "After sorting: ";
    bubble.printelemets();

    return 0;
}
```

OUTPUT:



```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\MANCHEM HIMESH\OneDrive\Desktop\DAA>
...
cd "c:\Users\MANCHEM HIMESH\OneDrive\Desktop\DAA\" ; if ($?) { g++ insertionsortexample.cpp -o
insertionsortexample } ; if ($?) { .\insertionsortexample }
Enter the size of the array: 6
Enter the elements: 65
34
8
16
33
25
Before sorting: 65 34 8 16 33 25
After sorting: 25 33 16 8 34 65
PS C:\Users\MANCHEM HIMESH\OneDrive\Desktop\DAA> □
```

TIME COMPLEXITY:

INSERTION

Best case

```

for (i = 1; i < n; i++)
{
    key = arr[i];
    j = i - 1;
    while (j >= 0 & arr[j] > key)
    {
        arr[j+1] = arr[j];
        j--;
    }
    arr[j+1] = key;
}

```

$T(n) = O(n)$

average case

pass 1 $\rightarrow n-1$ comparisons
pass 2 $\rightarrow n-2$ comparisons
 $\vdots$
pass $n-1 \rightarrow 1$ comparison

ignore constants

Average case $= O(n^2)$

worst case

worst case occurs when pivot is always smallest or largest.

$n + (n-1) + (n-2) + \dots + 1$

$T(n) = \frac{n(n+1)}{2}$

worst case $= O(n^2)$

MERGE SORT

ALGORITHM:

Merge sort:

Input = array elements
Output = sorted array

step-1: MERGE(arr, low, mid, high) then
 step-2: set $i \leftarrow \text{low}$, $j \leftarrow \text{mid} + 1$, $k \leftarrow \text{low}$
 step-3: while $i \leq \text{mid}$, and $j \leq \text{high}$ then
 step-4: if $\text{arr}[i] \leq \text{arr}[j]$ then
 $\text{temp}[k] = \text{arr}[i]$
 $i = i + 1$
 else
 $\text{temp}[k] = \text{arr}[j]$
 $j = j + 1$
 end if
 $k = k + 1$
 end while
 step-5: while $j \leq \text{high}$ then
 $\text{temp}[k] = \text{arr}[j]$
 $j = j + 1$
 $k = k + 1$
 end while
 step-6: while $i \leq \text{mid}$ then
 $\text{temp}[k] = \text{arr}[i]$
 $i = i + 1$
 $k = k + 1$
 end while
 step-7: for $i \leftarrow \text{low}$, $k \leftarrow 0$ to $i \leftarrow \text{high}$ then
 $\text{arr}[i] = \text{temp}[k]$
 end for

step 2: MERGE SORT(arr, low, high) then do
 if $\text{low} < \text{high}$ then do
 step 9: $\text{mid} = (\text{low} + \text{high}) / 2$
 MERGE-SORT(arr, low, mid)
 MERGE-SORT(arr, mid+1, high)
 MERGE(arr, low, mid, high)
 step 10: end if
 step 11: end for

CODE:

```
#include <iostream>
using namespace std;
class mergesort
{
public:
    int size, arr[100], i;
    void arrayinput()
    {
        cout << "Enter array size: ";
        cin >> size;

        cout << "Enter array elements:\n";
```

```
for (i = 0; i < size; i++)
{
    cin >> arr[i];
}

void printarray()
{
    for (i = 0; i < size; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}

void merge(int low, int mid, int high)
{
    int temp[100];
    int i = low, j = mid + 1, k = low;

    while (i <= mid && j <= high)
    {
        if (arr[i] <= arr[j])
        {
            temp[k] = arr[i];
            i++;
        }
        else
        {
            temp[k] = arr[j];
            j++;
        }
        k++;
    }

    while (i <= mid)
    {
        temp[k] = arr[i];
        i++;
        k++;
    }

    while (j <= high)
    {
        temp[k] = arr[j];
        j++;
        k++;
    }

    for (i = low; i <= high; i++)
    {
```

```
arr[i] = temp[i];
    }
}

void mergellogic(int low, int high)
{
    if (low < high)
    {
        int mid = (low + high) / 2;

        mergellogic(low, mid);
        mergellogic(mid + 1, high);

        merge(low, mid, high);
    }
}

int main()
{
    mergesort ms;

    ms.arrayinput();

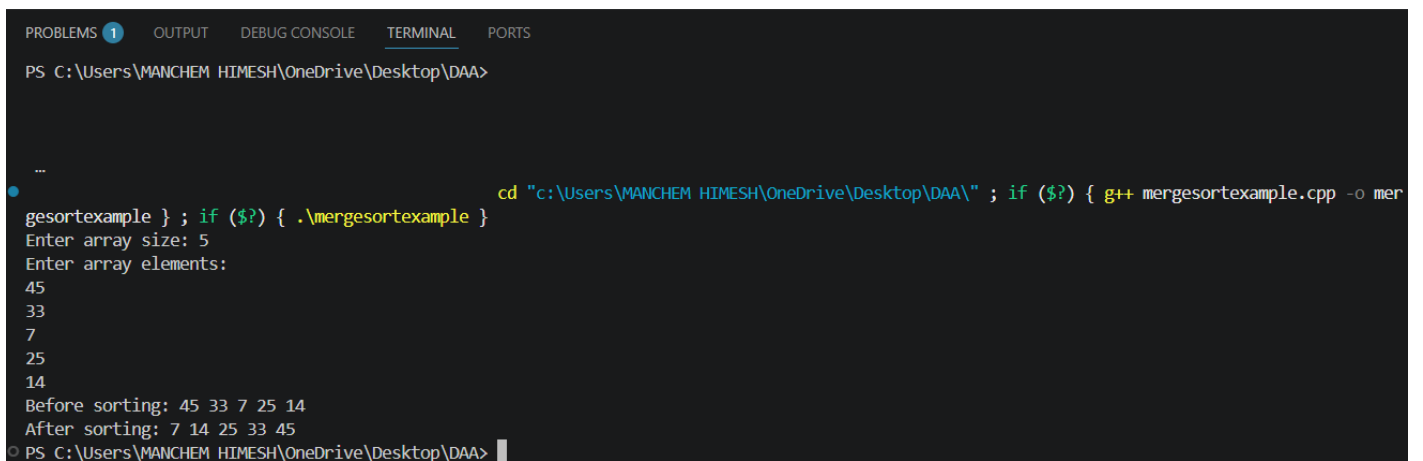
    cout << "Before sorting: ";
    ms.printarray();

    ms.mergellogic(0, ms.size - 1);

    cout << "After sorting: ";
    ms.printarray();

    return 0;
}
```

OUTPUT:



```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\MANCHEM HIMESH\OneDrive\Desktop\DAA>
...
cd "c:\Users\MANCHEM HIMESH\OneDrive\Desktop\DAA\" ; if ($?) { g++ mergesortexample.cpp -o mer
gesortexample } ; if ($?) { .\mergesortexample }
Enter array size: 5
Enter array elements:
45
33
7
25
14
Before sorting: 45 33 7 25 14
After sorting: 7 14 25 33 45
PS C:\Users\MANCHEM HIMESH\OneDrive\Desktop\DAA>
```

TIME COMPLEXITY:

Step Count / Frequency Count Method)

best case : Array is divided into $\log n$ levels and merging takes $O(n)$

```

Merge sort
void mergesort (int arr[], int low, int high)
{
    if (low < high)
    {
        mid = (low + high) / 2;
        mergesort (arr, low, mid);
        mergesort (arr, mid + 1, high);
        merge (arr, low, mid, high);
    }
}

void merge (int arr[], int low, int mid, int high)
{
    i = low;
    j = mid + 1;
    k = low;
    while (i <= mid && j <= high)
    {
        if (arr[i] < arr[j])
        {
            temp[k] = arr[i];
            i++;
        }
        else
        {
            temp[k] = arr[j];
            j++;
        }
        k++;
    }
    while (i <= mid)
    {
        temp[k] = arr[i];
        i++;
    }
    while (j <= high)
    {
        temp[k] = arr[j];
        j++;
    }
    k++;
    for (i = low; i <= high; i++)
    {
        arr[i] = temp[i];
    }
}
    
```

Recurrence Relation:

$$T(n) = 2T(n/2) + O(n)$$

Expand

$$T(n) = 2(2T(n/4) + n/2) + n$$

$$T(n) = 4T(n/4) + 2n$$

again

$$T(n) = 8T(n/8) + 3n$$

$$= 16T(n/16) + 4n$$

$$T(n) = 2^k T(n/2^k) + kn$$

$$n = 2^k$$

add log on both sides

$$T(n) = 2 \log n (1) + \log n (n)$$

$$T(n) = n \log n + n \log n$$

$$T(n) = n(1) + n \log n$$

$$T(n) = n \log n$$

average case :

Recurrence Relation:
 $T(n) = 2T(n/2) + O(n)$
 Expand
 $T(n) = 2(2T(n/4) + n/2) + n$
 $T(n) = 4T(n/4) + 2n$
 again
 $T(n) = 8T(n/8) + 3n$
 $= 16T(n/16) + 4n$
 $T(n) = 2^k T(n/2^k) + kn$
 $n = 2^k$
 add log on both sides
 $T(n) = 2 \log^2(n) + \log n(n)$
 $T(n) = n \log^2 + n \log n$
 $T(n) = n(n) + n \log n$
 $T(n) = n \log n$

worst case :

Recurrence Relation:
 $T(n) = 2T(n/2) + O(n)$
 Expand
 $T(n) = 2(2T(n/4) + n/2) + n$
 $T(n) = 4T(n/4) + 2n$
 again
 $T(n) = 8T(n/8) + 3n$
 $= 16T(n/16) + 4n$
 $T(n) = 2^k T(n/2^k) + kn$
 $n = 2^k$
 add log on both sides
 $T(n) = 2 \log^2(n) + \log n(n)$
 $T(n) = n \log^2 + n \log n$
 $T(n) = n(n) + n \log n$
 $T(n) = n \log n$

QUICK SORT

ALGORITHM:

Quick Sort:
 Input: array elements
 Output: sorted array
 Step 1: set pivot = arr[high]
 Step 2: set pivot = arr[high]
 Set i ← low - 1
 Step 3: for j ← low to high - 1 then
 Step 4: if arr[j] < pivot, then
 j ← i + 1
 swap arr[i] and arr[j]
 Step 5: end if
 Step 6: swap arr[i + 1] and arr[high]
 Step 7: return i + 1
 Step 8: quicksort (arr, low, high) then
 Step 9: if low < high then do
 pivot index = partition (arr, low, high)
 quicksort (arr, low, pivot index - 1)
 quicksort (arr, pivot index + 1, high)
 Step 10: end if
 Step 11: end for

CODE:

```
#include <iostream>
using namespace std;

class quicksort {
    int arr[100], size, i;

public:
    void getinput() {
        cout << "Enter the size of the array: ";
        cin >> size;

        cout << "Enter the elements: ";
        for (i = 0; i < size; i++)
            cin >> arr[i];
    }

    void printelements() {
```

```
for (i = 0; i < size; i++)
    cout << arr[i] << " ";
    cout << endl;
}

int partition(int lb, int ub) {
    int pivot = arr[lb];
    int start = lb;
    int end = ub;

    while (start < end) {
        while (start <= ub && arr[start] <= pivot)
            start++;

        while (arr[end] > pivot)
            end--;

        if (start < end)
            swap(arr[start], arr[end]);
    }

    swap(arr[lb], arr[end]);
    return end;
}

void quicksort1(int lb, int ub) {
    if (lb < ub) {
        int loc = partition(lb, ub);
        quicksort1(lb, loc - 1);
        quicksort1(loc + 1, ub);
    }
}

void sort() {
    quicksort1(0, size - 1);
}

};

int main() {
    quicksort quick;

    quick.getinput();

    cout << "Before Sorting: ";
    quick.printelements();

    quick.sort();

    cout << "After Sorting: ";
    quick.printelements();
}
```

```
return 0;
}
```

OUTPUT:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\MANCHEM HIMESH\OneDrive\Desktop\DAA> cd "c:\Users\MANCHEM HIMESH\OneDrive\Desktop\DAA\" ; if ($?) { g++ quicksort.cpp -o quicksort
} ; if ($?) { .\quicksort }
Enter the size of the array: 5
Enter the elements: 34
23
5
17
29
Before Sorting: 34 23 5 17 29
After Sorting: 5 17 23 29 34
PS C:\Users\MANCHEM HIMESH\OneDrive\Desktop\DAA> █
```

TIME COMPLEXITY : Step Count / Frequency Count Method)

best case : Pivot divides the array into nearly equal parts

Time Complexity:

$\text{quicksort}(arr, low, pivot_index-1); \rightarrow n/2$
 $\text{quicksort}(arr, pivot_index+1, high); \rightarrow n/2$
 $\text{partition}(arr, low, high); \rightarrow n$

$$T(n) = T(n/2) + T(n/2) + n$$

$$= 2T(n/2) + n$$

Recurrence Relation:

$$T(n) = 2T(n/2) + O(n)$$

Expand

$$T(n) = 2(2T(n/4) + n/2) + n$$

$$T(n) = 4T(n/4) + 2n$$

again

$$T(n) = 8T(n/8) + 3n$$

$$= 16T(n/16) + 4n$$

$$T(n) = 2^k T(n/2^k) + kn$$

$$n = 2^k$$

add log on both sides

$$T(n) = 2 \log_2 n (1) + \log_2 n (n)$$

$$T(n) = n \log_2 n + n \log_2 n$$

$$T(n) = n \log_2 n + n \log_2 n$$

$$T(n) = n \log_2 n$$

average case : Pivot usually creates reasonably balanced partitions

Recurrence Relation

$$T(n) = 2T(n/2) + O(n)$$

Expand

$$T(n) = 2(2T(n/4) + n/2) + n$$

$$T(n) = 4T(n/4) + 2n$$

again

$$T(n) = 8T(n/8) + 3n$$

$$= 16 + (n/16) + 4n$$

$$T(n) = 2^k (n/2^k) + kn$$

$$n = 2^k$$

add log on both sides

$$T(n) = 2 \log n (n) + \log n (n)$$

$$T(n) = n \log n + n \log n$$

$$T(n) = n \log n + n \log n$$

$$T(n) = n \log n$$

worst case : Pivot is always the smallest or largest element

Worst case:-

The worst case occurs when pivot always smallest or largest.

$$n(n-1) + n-2 + \dots + 1$$

$$T(n) = \frac{n(n-1)}{2}$$

Worst = $O(n^2)$

Conclusions

- 1. Bubble Sort:** Bubble Sort is a simple sorting algorithm that repeatedly compares adjacent elements and swaps them if they are in the wrong order. It is easy to implement but inefficient for large datasets, with $O(n^2)$ average and worst-case time complexity.
- 2. Selection Sort:** Selection Sort repeatedly finds the minimum element from the unsorted part and places it in the correct position. It is simple and performs fewer swaps, but its time complexity is $O(n^2)$.
- 3. Insertion Sort:** Insertion Sort builds the sorted array one element at a time by inserting each element into its correct position. It is efficient for **small or nearly sorted data**, with $O(n^2)$ average and worst-case complexity.
- 4. Merge Sort:** Merge Sort uses the **divide-and-conquer** technique to divide the array into smaller parts and then merge them in sorted order. It provides consistent $O(n \log n)$ time complexity and is suitable for large datasets.
- 5. Quick Sort:** Quick Sort uses the **divide-and-conquer** approach by selecting a pivot and partitioning the array around it. It is generally very fast, with $O(n \log n)$ average time complexity, although its worst case can be $O(n^2)$.